Vul4Java: A Java OSS vulnerability identification method based on a two-stage analysis

Zhuo Wang
School of Software, Henan University, Kaifeng, Henan, 475000, China; Henan International Joint Laboratory of Intelligent Network Theory and Key Technology, Henan University, Kaifeng, Henan, 475000, China
wangzhuo2022@iie.ac.cn

Jiahao Hu
School of Software, Henan University, Kaifeng, Henan, 475000, China; Henan International Joint Laboratory of Intelligent Network Theory and Key Technology, Henan University, Kaifeng, Henan, 475000, China

Yijun Zhou
School of Software, Henan University, Kaifeng, Henan, 475000, China; Henan International Joint Laboratory of Intelligent Network Theory and Key Technology, Henan University, Kaifeng, Henan, 475000, China

Sidy Tambadou
School of Software, Henan University, Kaifeng, Henan, 475000, China; Henan International Joint Laboratory of Intelligent Network Theory and Key Technology, Henan University, Kaifeng, Henan, 475000, China
stambadou@groupeisi.com

Fang Zuo *
School of Software, Henan University, Kaifeng, Henan, 475000, China; Henan International Joint Laboratory of Intelligent Network Theory and Key Technology, Henan University, Kaifeng, Henan, 475000, China
zuofang@henu.edu.cn

ABSTRACT

Open source software (OSS) has been widely used to accelerate software development, inevitably exposing downstr omissions and false positives; omissions put applications and their users at risk, and false positives increase the burden on software developers and users. Therefore, in this paper, we propose a two-phase based approach for JAVA OSS vulnerability analysis. To reduce underreporting, we construct a comprehensive third-party library and vulnerability association database; to avoid false positives, first, we use a static analysis method to extract the structure-aware call graph (SACG) of oss and locate the information of the vulnerability functions; second, we compare the similarity between the OSS vulnerability functions and the pre-patch and post-patch functions based on the vulnerability patches to determine the existence of the patches in the OSS, and then verify the OSS vulnerability information. We evaluate the method on a dataset of 7 open source projects and 167 vulnerability information, and the F1 value of the method is 0.779, which is higher than the currently available SOTA tools.

CCS CONCEPTS

• **Security and privacy** → Software and application security; Software security engineering.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASENS 2024, April 26–28, 2024, Nanchang, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0978-4/24/04
https://doi.org/10.1145/3677182.3677315

KEYWORDS

Java OSS, Vulnerability, Patch Analysis

ACM Reference Format:
Zhuo Wang, Jiahao Hu, Yijun Zhou, Sidy Tambadou, and Fang Zuo *. 2024. Vul4Java: A Java OSS vulnerability identification method based on a two-stage analysis. In *International Conference on Algorithms, Software Engineering, and Network Security (ASENS 2024), April 26–28, 2024, Nanchang, China.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3677182.3677315

1 INTRODUCTION

The OSS ecosystem has seen a rise in popularity of the OSS-based secondary development model. However, this model has been associated with several security issues. Synopsys' 2022 report states that 97% of codebases use third-party open source code, particularly in the computer hardware, cybersecurity, energy technology, and IoT sectors [1]. The Java ecosystem, and in particular its primary third-party component repository, Maven-Central, has shown significant growth, hosting more than 11.98 million versions of third-party libraries as of December 2023 [2].

However, the number of disclosed vulnerabilities in OSS has been steadily increasing since 2009, posing a threat not only to the libraries themselves but also to the downstream projects that depend on them. For instance, over 35,000 Java packages in the Maven ecosystem were affected by a vulnerability in Apache Log4j2. OWASP lists 'vulnerable and outdated components' as one of the main cybersecurity risks due to this trend [3].

There are two main categories of existing vulnerability detection methods: metadata-based [3, 4, 5] and code-based [6-10]. Metadata-based methods rely on information from software configuration files. However, in the absence of such metadata, relevant third-party libraries and vulnerabilities cannot be identified. Code-based methods are more effective. However, different vulnerability databases can lead to underreporting and false positives.

To tackle these issues, we propose a new approach that focuses on detecting vulnerabilities in Java OSS. This approach involves two steps: (1) constructing mapping relationships between Java OSS and vulnerability information in publicly available vulnerability databases, and (2) using static analysis techniques to extract SACGs and identify vulnerability functions based on vulnerability patch information. The vulnerability information is verified by comparing the similarity of the functions in the SACG with the code features before and after the vulnerability patches. The method was evaluated on a dataset containing 7 open source projects and 167 vulnerability information. The F1 value of this method is 0.779, which is higher than the existing State of The Art (SOTA) tool.

The contributions of this paper are summarized as follows:

- For empirical analysis and evaluation, we collected a dataset of 373 binary versions and 167 vulnerability reports from seven Java open-source projects.
- We propose a two-stage vulnerability detection method. To validate false positives in associated vulnerabilities, we use SACG to locate the vulnerability function. We then identify vulnerability information by comparing the similarity of the vulnerability function before and after patching.
- We compared our method with state-of-the-art vulnerability detection methods and found that it achieved a higher F1 value (0.779) and more reliable vulnerability detection results.

2 RELATED WORK

There are two main categories of existing vulnerability detection tools: metadata-based and code-based. Two metadata-based approaches for identifying vulnerabilities in software are Dependency-Check [3] and OpenSCA [4]. Dependency-Check extracts information about dependencies used in the software and matches them with CVE-related CPEs. OpenSCA is a comprehensive security vetting tool that combines open-source intelligence and code analysis techniques to identify third-party components and their associated vulnerabilities in software. Additionally, Dependabot [4] is an automated tool that monitors project dependencies and provides alerts and fix recommendations when vulnerabilities or available updates are found. However, these methods rely on information from software configuration files, such as POM.XML, and are limited in their effectiveness when metadata is missing or interfered with by code obfuscation techniques, such as repackaging or package flattening.

On the other hand, there are code-based approaches such as VERJava [6], Eclipse Steady [7], and JFinder [9]. VERJava uses a two-phase approach to accurately identify vulnerable versions of Java OSS. Eclipse Steady combines static and dynamic analysis techniques to provide a comprehensive vulnerability detection and assessment framework. However, it relies on a self-built database that may lead to low recall. JFinder utilises an innovative architecture that combines quaternary self-concern and pre-training mechanisms to identify security vulnerabilities in Java code. However, its analyses are limited to specific types of vulnerabilities and are ineffective when faced with code obfuscation.

Overall, code-based approaches are generally considered more effective [8]. However, it is important to note that these tools have limitations, such as dependence on specific databases, inefficient processing, or limited detection of certain types of vulnerabilities. Therefore, further research is needed to improve the accuracy and efficiency of vulnerability detection.

3 APPROACH AND DESIGN

This section provides a detailed introduction to our method. The input of our method is an OSS in binary format. Firstly, we identify vulnerability information of the OSS using the constructed vulnerability database. Secondly, we validate the vulnerability based on the vulnerability information of the OSS and the patch file. The details of the vulnerability correlation and validation are explained below.

3.1 Vulnerability Association

The objective of this section is to create a mapping relationship table for OSS and vulnerability associations. To achieve this, we first gather vulnerability data from publicly available databases such as CVE [11], and Github [12]. Next, we construct a mapping relation table by associating the collected OSS information with the vulnerability information [13].

To ensure the recall of our method, we aim to collect as much vulnerability data as possible. Therefore, we consider the following aspects when collecting vulnerability data: To collect data on vulnerable OSS versions, we convert the names of the collected OSSs to Common Platform Enumeration (CPE) [14] format. We then use the professional CVE search tool cve-search [15] to query public CVE databases for vulnerable OSSs by mapping the converted OSS names. This method allows us to obtain information on known vulnerabilities of an OSS, including CVE number, vulnerability type, vulnerability description, Common Vulnerability Scoring System (CVSS) score, and vulnerable version. This paper employs CVSS v3.0 to indicate the severity of the collected vulnerabilities. A total of 1013 CVE vulnerability information was collected from 266 unique OSSs.

Supplementing Vulnerability Information. To ensure the recall of our method, we expand the collected vulnerability information. We supplement the collected vulnerability information based on the name and version of the third-party libraries in the NVD database and Github. Finally, we concatenate all the correlated vulnerability information as the final result, which is correlated to a total of 1,203 pieces of vulnerability information.

3.2 Vulnerability Verification

Using the vulnerability association information, we employed a two-stage verification method to confirm the associated vulnerabilities. The method's specific flow is illustrated in Figure 1. The first step involves using static analysis to extract the SACG of the OSS. Then, vulnerability patch information is used to locate the list of vulnerability functions. The vulnerability information is validated by determining whether a vulnerability function is contained in the SACG. In the second step, we compare the code features of the OSS with those of the vulnerability functions before and after the vulnerability patches. This helps us determine whether a patch exists in the OSS as a result of vulnerability verification in OSS.

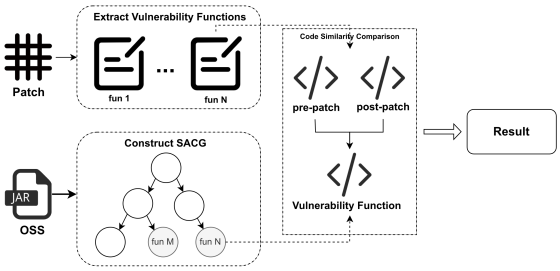

Figure 1: The Workflow of Vulnerability Verification.

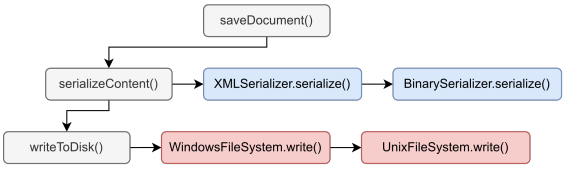

Figure 2: Part of the Java-OpenPDF SACG.

3.2.1 Structure-aware call graph verification. This study employs a static analysis approach to generate SACGs that comprehensively capture explicit and implicit method calls in a program. The initial step involves extracting an initial function call relation graph from the jar file using the Spark algorithm in the Soot framework. However, traditional static call graphs (CG) may miss certain calls due to the lack of full support for dynamic program features. To enhance the call graphs, we introduce Class Hierarchy Analysis (CHA) which reveals the inheritance structure of the classes and the potential overriding relationships of the methods. This is especially useful in dealing with polymorphic OSS.

To enhance the precision of the call graph, this study incorporates Rapid Type Analysis (RTA) and Variable Point Analysis. These analyses assist in limiting the scope of potential call targets and more accurately forecasting and identifying potential call relationships in third-party libraries by determining the type instances created in the program and how they interact.

As an example, we compare the outcomes of the analyses of traditional CG and SACG using the Java-OpenPDF library. In traditional computer graphics, the call path of the saveDocument() method is relatively simple. However, with the addition of SACG, along with CHA and variable point analysis, a more complex situation is revealed. For instance, it has been discovered that serializeContent() is an interface implemented by multiple subclasses, and writeToDisk() invokes various file system APIs that rely on the operating system and user configuration. Therefore, SACG offers a more comprehensive and detailed call graph that accurately captures the program's dynamic behaviour and additional function call relationships.

The Figure 2 displays the function call relationship graph constructed by the normal CG method, with grey nodes representing the original graph. The blue nodes represent the function call relationships added by applying CHA, and the red nodes represent the function call relationships added by applying variable point analysis. The analysis results show the multiple method calls that may occur due to different document types and operating systems, as well as the potential impact of user settings on the call paths. Traditional CG methods do not reveal these complexities and dynamics due to class hierarchies and runtime variables.

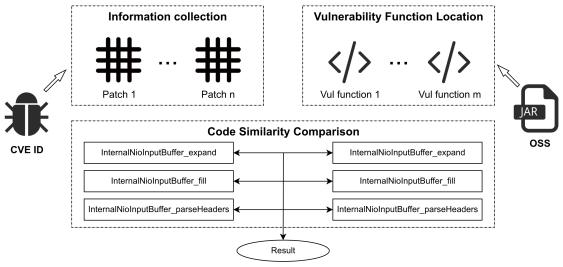

Figure 3: Framework for Patch Verification.

3.2.2 Vulnerability patch verification. The aim of this section is to confirm the presence of a vulnerability patch in a given JAR file. The process consists of three main steps: information gathering, locating the vulnerability function, and comparing code similarity. The framework for patch verification can be seen in Figure 3.

Information gathering involves using a custom crawler to collect CVE patches and multiple versions of source code. To obtain CVE patches, we linked directly from the NVD's CVE description page to the GitHub or SVN patch page, cross-searched the project's issue list to locate fix commits, or searched for CVE IDs in the project's issue list to locate relevant patches. We then split each patch obtained into functions, extracted details, and stored pre- and post-patch versions in JSON format.

To enhance the efficiency and accuracy of decompiling JAR files, automated scripts were developed using the functionality of JD-GUI [16]. The Python script accepts directory input from the JAR file, navigates and decompiles the class files using JD-GUI, and securely stores the source code in the local file system.

The vulnerability function location is followed by a code similarity comparison. During this phase, we identify and compare vulnerable functions in third-party libraries and their code before and after patching. We use various comparison methods, including structural and logical flow analyses such as Abstract Syntax Trees (ASTs) or machine learning-based models, in addition to text comparisons. We assign a similarity score to each function to determine its similarity to the pre and post-patch versions. If the function resembles the pre-patch version more closely, it is possible that the patch was not applied, and a security vulnerability still exists. If it resembles the post-patch version more closely, the vulnerability may have been resolved.

4 EVALUATION

This section aims to describe the process of constructing the thesis dataset and evaluate the effectiveness of the proposed method through experiments on this dataset.

4.1 Dataset

The dataset used in this study differs from that of the VERJava paper, which focuses on the source code of Java open source projects. Our study focuses on the binary file format of these projects, specifically

Table 1: Detailed description of the dataset.

Target	CVEs	Commits	Functions	Versions
Tomcat	45	125	430	144
Jackson-databind	62	68	104	46
Structs	14	18	114	41
Jenkins	16	21	61	72
Liferay-portal	20	99	200	13
Spring-security	3	3	44	15
Spring-framework	7	10	107	42
Total	167	344	1060	373

JAR files. We collected JAR files of all versions of seven open source projects in Maven, resulting in a total of 373 J-files. Please refer to Table 1 for a detailed description of the dataset.

4.2 Effectiveness

The objective of this study is to assess the efficacy of vulnerability detection techniques for OSS that target binary file formats. This will be achieved by comparing them with tools such as Dependency-Check, Eclipse Steady, and OpenSCA. The evaluation metrics used are TP (True Positive), FP (False Positive), and FN (False Negative). TP refers to accurately identified vulnerabilities, FP refers to incorrectly labelled non-affected vulnerabilities, and FN refers to unidentified vulnerabilities.

Table 2: Vulnerability Detection Results.

Tools	TP	FP	FN	P	R	F1	Accu.
Dependency-Check	**243**	956	**4**	0.203	**0.984**	0.337	0.210
Eclipse Steady	157	473	90	0.249	0.636	0.358	0.207
OpenSCA	230	968	17	0.192	0.931	0.318	0.189
Vul4Java	213	**86**	34	**0.712**	0.862	**0.779**	**0.637**

After comparing the performance of four different tools (Dependency-Check, Eclipse Steady, OpenSCA, and Vul4Java) on the vulnerability detection task, we observed the strengths and limitations of each tool, as shown in Table 2. Dependency-Check and OpenSCA showed high sensitivity in capturing vulnerabilities, but also produced more false positives. Vul4Java performed the best in terms of accuracy with an accuracy of 0.712 and achieved a high F1 score (0.779) in balancing accuracy and recall. Although Eclipse Steady performed well in terms of recall, its accuracy was lower than the other tool, resulting in the review team needing to take extra time to validate each report. The other tool had a slightly lower number of true positives but showed a good balance between true positives and false positives with an overall accuracy of 0.637.

Overall, while each tool has its own characteristics and limitations, Vul4Java demonstrated the highest accuracy, F1 score, and overall accuracy, indicating potential for vulnerability detection tasks. This suggests that a combination of tools may be necessary to enhance the comprehensiveness and accuracy of vulnerability detection in real-world applications.

5 CONCLUSIONS

This paper presents Vul4Java, an approach for identifying vulnerabilities in Java OSS through a two-stage vulnerability analysis. The vulnerability information disclosed by OSS is correlated. The existence of paths to reach the vulnerability functions is determined based on the SACG of OSS. The existence of patches in the OSS is determined by comparing the similarity of the code before and after the vulnerability functions and the vulnerability patches, and then determining the existence of vulnerabilities in the OSSs. The method was evaluated on a dataset of 373 versions of binaries from 7 Java open-source projects and 167 vulnerability messages. The achieved F1 value of 0.779 is significantly higher than that of other existing vulnerability detection methods.

ACKNOWLEDGMENTS

This work was supported in part by Postgraduate Education Reform and Quality Improvement Project of Henan Province, China (Grant No.YJS2022JD26), Graduate Education Innovation and Quality Improvement Project of Henan University, China (Grant No.SYLJD2022008, SYLKC2022028), 2023 Graduate Education and Teaching Reform Research and Practice Project of Henan University, China (No.YJSJG2023XJ021), Henan Province Excellent Foreign Expert Introduction Project, China (Grant No.HNGD2023005), Key R&D and Promotion Special Project of Henan Province, China(Grant No.222102210055).

REFERENCES

[1] Snky. 2023. https://snyk.io/.
[2] Maven-Central. 2023. https://mvnrepository.com/repos/central.
[3] Dependency-Check. 2023. https://owasp.org/www-project-dependency-check.
[4] Dependabot. 2023. https://docs.github.com/en/code-Security/dependabot/dependabot-alerts.
[5] OpenSCA. 2022. https://opensca.xmirror.cn/.
[6] Q. Sun et al. 2022. "VERJava: Vulnerable Version Identification for Java OSS with a Two-Stage Analysis," 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), Limassol, Cyprus, pp. 329-339, https://doi: 10.1109/ICSME55016.2022.00037.
[7] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2020. Detection, assessment and mitigation of vulnerabilities in open source dependencies. Empirical Softw. Engg. 25, 5, pp.3175-3215. https://doi.org/10.1007/s10664-020-09830-x.
[8] N. Saccente, J. Dehlinger, L. Deng, S. Chakraborty and Y. Xiong. 2019. "Project Achilles: A Prototype Tool for Static Method-Level Vulnerability Detection of Java Source Code Using a Recurrent Neural Network," 2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW), San Diego, CA, USA, pp. 114-121, https://doi: 10.1109/ASEW.2019.00040.
[9] J. Wang *et al.* 2023. "JFinder: A novel architecture for java vulnerability identification based quad self-attention and pre-training mechanism," High-Confidence

Computing, Volume 3, Issue 4, pp. 2667-2952, https://doi.org/10.1016/j.hcc.2023.100148.

[10] L. Zhao *et al.* 2023. Software Composition Analysis for Vulnerability Detection: An Empirical Study on Java Projects. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Sympo-sium on the Foundations of Software Engineering (ESEC/FSE '23), New York, NY, USA, pp. 960–972, https://doi.org/10.1145/3611643.3616299.

[11] CVE. 2023. https://cve.mitre.org/.

[12] GitHub. 2023. https://github.com/.

[13] L. Zhang et al. 2023. "Mitigating Persistence of Open-Source Vulnerabilities in Maven Ecosystem," 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), Luxembourg, Luxembourg, pp. 191-203, https://doi:10.1109/ASE56229.2023.00058.

[14] CPE. 2023. https://nvd.nist.gov/Products/CPE.

[15] CVE-search. 2020. https://github.com/cve-search/cve-search.

[16] JD-GUI. 2023. https://github.com/java-decompiler/jd-gui.